CAN Bus API

Development Documentation

V1.1.0

Update Records

Date	Description
2022-10-31	First Edition Release
2023-02-22	Modify the API
2023-04-23	Update the API
2024-06-20	Update API and documentation

Contents

1 Introduction

1.1 Frame Format

Frame Format	Description
CanUtil.STANDARD_FRAME_FORMAT	Standard Frame
CanUtil.EXTEND_FRAME_FORMAT	Extend Frame

1.2 Frame Type

Frame Type	Description
CanUtil.DATA_FRAME_TYPE	Data Frame
CanUtil.REMOTE_FRAME_TYPE	Remote Frame

1.3 CanRecvInfo Class

Package: com.common.apiutil.can
CAN receives information classes and can set or get property values through the set and get methods.

```
private String recvId;      // Receive ID
private int recvDataLen;    // Data length
private byte[] recvData;    // Data
private int frameFormat;    // Frame format
private int frameType;      // Frame type
```

1.4 CanUtil Class

1.4.1 Open the CAN Bus

```
/**
 * @Function Open the CAN Bus
 *
 * @Parameter
 * 1.bitRate: Bit rate
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int open(int bitRate)
```

1.4.2 Close the CAN Bus

```
/**
 * @Function Close the CAN Bus
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int close()
```

1.4.3 Get the Status of the CAN Bus

```
/**
 * @Function Get the status of the CAN bus
 *
 * @Return 0, Close; 1, Open
 *
 * */

public static int getStatus()
```

1.4.4 Set the Bit Rate

```
/**
 * @Function Set the bit rate
 *
 * @Parameter
 * 1.bitRate: value
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int setBitRate(int bitRate)
```

1.4.5 CAN Bus Receive Data

```
/**
 * @Function CAN receive data
 *
 * @Parameter
 * 1.canRecvInfo: A data object that contains the received ID and data frame
 *
 * */

public interface CanOperationListener
```

```
{
    void canInput(CanRecvInfo canRecvInfo);
}
```

1.4.6 CAN Bus Receive

```
/**
 * @Function CAN bus receive
 *
 * @Parameter
 * 1.canOperationListener: Receive callback
 *
 * */

public static void canRecv(CanOperationListener canOperationListener)
```

1.4.7 CAN Bus Send Frames

(standard frame/extended frame/data frame/remote frame)

```
/**
 * @Function CAN Bus Send Frames
 *
 * @Parameter
 * 1.frameFormat:
 *      1.CanUtil.STANDARD_FRAME_FORMAT(standard frame)
 *      2.CanUtil.EXTEND_FRAME_FORMAT(extended frame)
 * 2.frameType:
 *      1.CanUtil.DATA_FRAME_TYPE(data frame)
 *      2.CanUtil.REMOTE_FRAME_TYPE(remote frame)
 * 3.id: ID Number
 * 4.data: Data Frame, Remote Frame(0x00)
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * @CommonException
 * Parameter exception, ID is not hexadecimal and starts at 0, or the frame format
is not a standard frame or extended frame, or the frame type is not a data frame
or a remote frame
 *
 * */

public static int canSend(int frameFormat, int frameType, String id, byte[] data)
throws CommonException
```

1.5 CanUtil2 Class

Package: com.common.apiutil.can.CanUtil2
Can Control class

1.5.1 Constant

Constant	Value	Description
CanUtil2.CAN_ID_STANDARD	0	Standard ID Frame
CanUtil2.CAN_ID_EXTENDED	4	Extended ID Frame
CanUtil2.CAN_RTR_DATA	0	Data Frame
CanUtil2.CAN_RTR_REMOTE	2	Remote Frame

1.5.2 Data frame

(Take a frame of CAN data frame as an example)

Data bit	Description
data[0], data[1], data[2], data[3]	4 bytes of the standard ID
data[4], data[5], data[6], data[7]	4 bytes of the extension ID
data[8]	1 byte frame type, indicating whether the data frame is a standard frame or an extended frame
data[9]	1 byte frame type, indicating whether it is a data frame or a remote frame
data[10]	1 byte length, indicating the effective data length in the frame
data[11], data[12], data[13], data[14], data[15], data[16], data[17], data[18]	8 bytes of data bits
data[19]	1 byte FMI

Dataframe parsing example:

```
CanUtil2 can = new CanUtil2(getApplicationContext());
byte[] ret = can.read(20, 5);
if (ret.length < 20) {
    return;
}

int id;

if (ret[8] == CanUtil2.CAN_ID_STANDARD) {
```

```
    // Standard ID
    id = (((int) ret[3] & 0xff) << 24) + (((int) ret[2] & 0xff) << 16) +
        (((int) ret[1] & 0xff) << 8) + (((int) ret[0] & 0xff));
} else if (ret[8] == CanUtil2.CAN_ID_EXTENDED) {
    // Extended ID
    id = (((int) ret[7] & 0xff) << 24) + (((int) ret[6] & 0xff) << 16) +
        (((int) ret[5] & 0xff) << 8) + (((int) ret[4] & 0xff));
}

if (ret[9] == CanUtil2.CAN_RTR_DATA) {
    // Data frame
    String dataString = new String(ret, 11, 8, StandardCharsets.UTF_8);
} else if (ret[9] == CanUtil2.CAN_RTR_REMOTE) {
    // Remote frame
}
```

1.5.3 Set Can working mode

```
/**
 * @Function Configure CAN working mode
 *
 * @Parameter
 * 1.CAN_SJW: Resync skip width (SJW), Range: [0, 3]
 * 2.CAN_BS1: Time1, Range: [0, 15]
 * 3.CAN_BS2: Time2, Range: [0, 7]
 * 4.CAN_Prescaler: Range: [1, 1024]
 * 5.CAN_MODE: Normal mode or Loop mode
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int configMode(int CAN_SJW, int CAN_BS1, int CAN_BS2, int
CAN_Prescaler, int CAN_MODE)
```

Baud rate calculation formula:

Baud Rate=108×1000000/"CAN_Prescaler"/("CAN_SJW"+"CAN_BS1"+"CAN_BS2"+3)

1.5.4 Can filter mode

```
/**
 * @Function Configure CAN filter mode
 *
 * @Parameter
 * 1.CAN_FilterNumber: Filter number, Range: [0, 13]
 * 2.CAN_FilterMode: Mode, {@code 0} Shielded bit mode; {@code 1} Identifier list
mode
```

```
 * 3.CAN_FilterScale: Filter Scale{@code 0} 16 bits;{@code 1} 32 bits
 * 4.CAN_FilterIdHigh: Used to store the ID to be filtered, Range: [0, 65535]
 * 5.CAN_FilterIdLow: Used to store the ID to be filtered, Range: [0, 65535]
 * 6.CAN_FilterMaskIdHigh: Used to store the ID or mask to filter, Range: [0,
65535]
 * 7.CAN_FilterMaskIdLow: Used to store the ID or mask to filter, Range: [0,
65535]
 * 8.CAN_FilterActivation: {@code 0} Close；{@code 1} Open
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int configFilter(
    int CAN_FilterNumber,
    int CAN_FilterMode,
    int CAN_FilterScale,
    int CAN_FilterIdHigh,
    int CAN_FilterIdLow,
    int CAN_FilterMaskIdHigh,
    int CAN_FilterMaskIdLow,
    int CAN_FilterActivation)
```

1.5.5 Send the Can message

```
/**
 * @Function Write information to Can bus
 *
 * @Parameter
 * 1.ID: Message ID, Range:[0, 65535]
 * 2.IDE:
 *      1.CanUtil2.CAN_ID_STANDARD
 *      2.CanUtil2.CAN_ID_EXTENDED
 * 3.RTR:
 *      1.CanUtil2.CAN_RTR_DATA
 *      2.CanUtil2.CAN_RTR_REMOTE
 * 4.data: data value
 * 5.len: data length
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int write(int ID, int IDE, int RTR, byte[] data, int len)
```

1.5.6 Read the Can message

```
/**
 * @Function Read data from Can bus
```

```
     *
     * @Parameter
     * 1.len: Read data length (recommended to read in multiples of 20)
     * 2.timeout: timeout waiting time
     *
     * @Return ResultCode: please refer the quick guidance document.
     *
     * */

    public synchronized byte[] read(int len, int timeout)
```